

Module 01: Memory Hierarchy and Address Binding

1. Memory Hierarchy & Tradeoffs

Computer system me CPU speed aur secondary storage ke latency gap ko bridge karne ke liye hierarchy use hoti hai: Registers (<1ns) → Caches (1-10ns) → RAM (50-100ns) → NVMe/SSD (μs) → HDD (ms). Upar se neeche jane par latency badhti hai aur cost per bit ghat ti hai.

2. Bare Machine vs Resident Monitor

Bare Machine: Shuruati systems me zero memory protection tha; user directly hardware ko manipulate karta tha.

Resident Monitor: OS ka first avatar jisme fence register use karke memory ko OS space aur user program space me protect kiya gaya.

3. Logical vs Physical Address Space

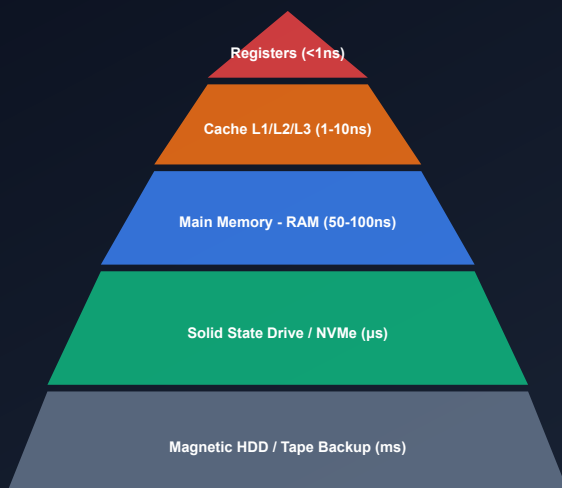
Feature	Logical (Virtual) Address	Physical Address
Generator	CPU execution engine	Memory unit (RAM bus)
User View	0 se MAX tak continuous flat address space	Actual hardware silicon cell address
Translation	MMU dwara runtime par translate hota hai	Directly memory bus par transmit hota hai

4. Address Binding: 3 Phases

- Compile Time:** Absolute addresses generate hote hain. Program relocate nahi ho sakta.
- Load Time:** Relocatable addresses generate hote hain. RAM me load hote hi base address fix ho jata hai.
- Execution (Run) Time:** Dynamic relocation register hardware (MMU) use karta hai. Process runtime par RAM me move ho sakta hai.

Address Binding Phases & Dynamic Relocation

Memory Hierarchy Pyramid



3-Stage Address Binding Flow

1. Compile Time Binding

Generates Absolute Code. Agar starting location badle to recompile karna padta hai.
Example: MS-DOS .COM programs, embedded microcontrollers.



2. Load Time Binding

Generates Relocatable Code. Compiler symbolic offset generate karta hai; Loader memory me load karte waqt final base address bind karta hai.



3. Execution (Run) Time Binding (Modern OS)

Logical addresses runtime par Physical addresses me translate hote hain.

Hardware Support: MMU (Memory Management Unit)

Formula: $\text{Physical Address} = \text{Logical Address} + \text{Relocation Register}$

Allows process to move dynamically in RAM during execution!

5. Dynamic Loading vs Dynamic Linking

Dynamic Loading: Routines tabhi memory me aate hain jab unhe invoke kiya jaye (memory bachti hai).

Dynamic Linking: Shared libraries (.so / .dll) ek baar RAM me load hoti hain aur multiple processes runtime stubs ke through execute karte hain.

6. Solved Numerical

Problem: Base = 14000, Limit = 2500.

1. Logical Address 1240: Check $1240 < 2500$ (Valid). Physical Address = $14000 + 1240 = 15240$.

2. Logical Address 2750: Check $2750 \geq 2500$ (Invalid). Hardware MMU triggers **Trap to OS: Memory Access Violation**.

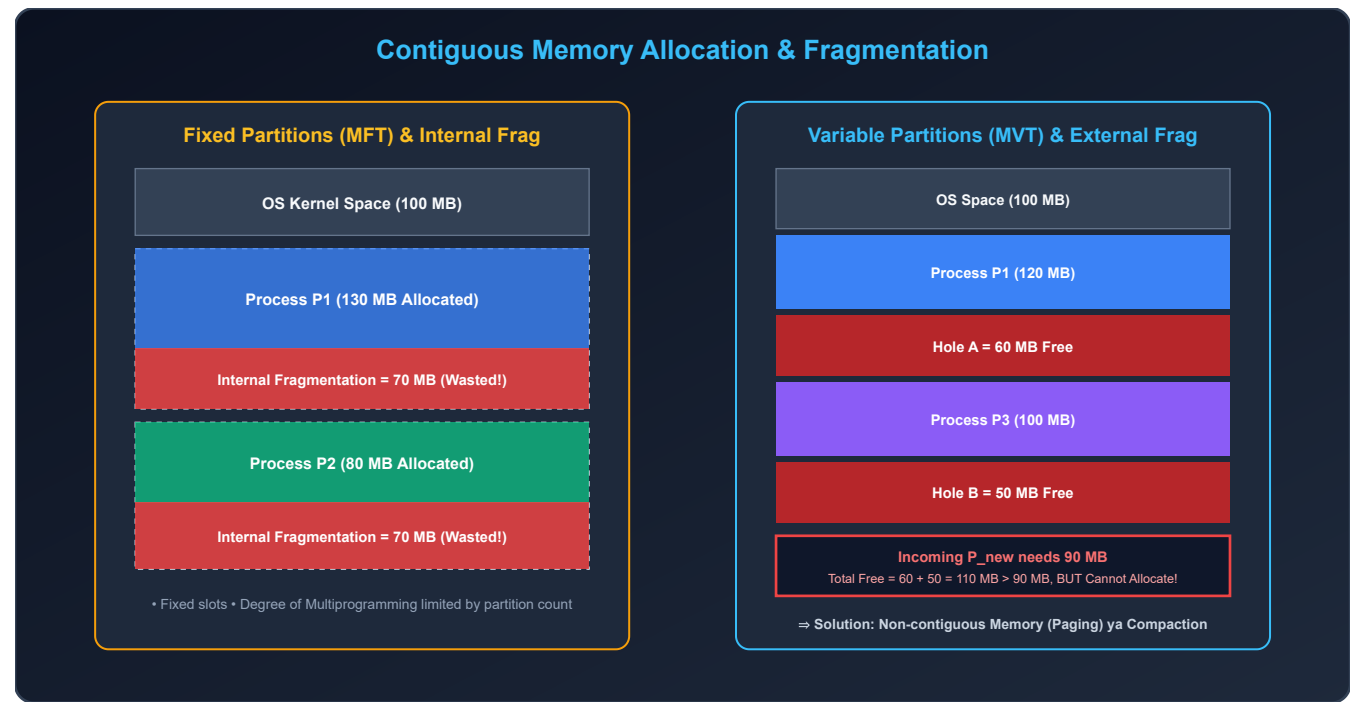
Module 02: Contiguous Allocation and Fragmentation

1. Contiguous Allocation Concept

Contiguous Allocation me har process ko RAM me ek unbroken continuous block of memory addresses assign hota hai.

2. Fixed Partition (MFT) vs Variable Partition (MVT)

Parameter	Fixed Partitions (MFT)	Variable Partitions (MVT)
Partition Size	Static / Boot time par predefined	Dynamic / Process requirement ke mutabik
Degree of Multiprogramming	Fixed (Number of partitions ke barabar)	Dynamic (Jitne process fit ho sakein)
Primary Fragmentation	Internal Fragmentation	External Fragmentation
Hardware Overhead	Very Low	Hole management table overhead



3. Internal vs External Fragmentation

Internal Fragmentation: Partition process se bada hota hai, partition ke andar unused memory waste hoti hai.
External Fragmentation: Total free memory sufficient hai, lekin scattered hone ki wajah se process allocate nahi ho pata (50% rule applies).

4. Hardware Protection: Base & Limit Registers

CPU logical address L generate karta hai. Agar $L < \text{Limit}$, tab physical address = $L + \text{Base}$. Otherwise, MMU ek trap generate karta hai (Segmentation fault).

Module 03: Dynamic Storage Allocation Strategies

1. Allocation Strategies Overview

Strategy	Algorithm Rule	Search Overhead	Key Advantage / Disadvantage
First Fit	Pehla hole jo size satisfy kare	Fast ($O(1)$ to $O(n)$)	Fastest; early memory fragmentation
Best Fit	Smallest hole jo size satisfy kare	Strictly $O(n)$	Preserves large holes; creates tiny useless slivers
Worst Fit	Largest available hole	Strictly $O(n)$	Large leftover; destroys largest blocks quickly
Next Fit	Last allocated pointer se circular First Fit	Fast	Spreads allocation evenly

Dynamic Storage Allocation Strategies: Comparison

1. First Fit

Working Principle:

Pehle hole se search shuru karo; jo PEHLA hole process size se bada mila, allocate kar do.

Advantages:

• Fastest execution speed

- Poori list search nahi karni padti
- Disadvantages: • Shuruati memory blocks me tiny unusable holes jama ho jate hain. Time Complexity: $O(n)$ worst / $O(1)$ best
2. Best Fit Working Principle: Poori list search karo aur wo hole chuno jo process se bada ho aur SMALLEST ho. Advantages: • Badi holes preserve rehti hain future ke bade processes ke liye. Disadvantages: • Generates smallest leftover holes jo kisi kaam ke nahi hote (Worst fragmentation). Time Complexity: Strictly $O(n)$
3. Worst Fit Working Principle: Poori list me se LARGEST available hole ko choose karo aur allocate karo. Advantages: • Leftover hole ka size bada hota hai, jisme naya process fit ho sakta hai. Disadvantages: • System ke largest holes jaldi khatam ho jate hain; bade jobs starve karte hain. Time Complexity: Strictly $O(n)$
4. Next Fit & Compact Next Fit Strategy: First Fit ki tarah, lekin search start se nahi balki LAST ALLOCATION pointer se hoti hai. Compaction (Defra): External fragmentation khatam karne ke liye saare processes ko memory ke ek side shift karna taaki ek bada contiguous free block ban jaye. Compaction Condition: Possible ONLY if Dynamic Relocation is supported at runtime!

2. Solved Numerical Summary

Holes: 100K, 500K, 200K, 300K, 600K | **Processes:** P1(212K), P2(417K), P3(112K), P4(426K)

- **First Fit:** P1 in 500K, P2 in 600K, P3 in 500K remaining, **P4 Waits (Cannot Allocate).**
- **Best Fit:** P1 in 300K, P2 in 500K, P3 in 200K, P4 in 600K → **All 4 Processes Successfully Allocated!**
- **Worst Fit:** P1 in 600K, P2 in 500K, P3 in 600K remaining, **P4 Waits (Cannot Allocate).**

3. Compaction (Defragmentation)

External fragmentation ko khatam karne ke liye saare processes ko relocate karke contiguous hole banaya jata hai. Iske liye **Execution Time Dynamic Relocation** mandatory hota hai.

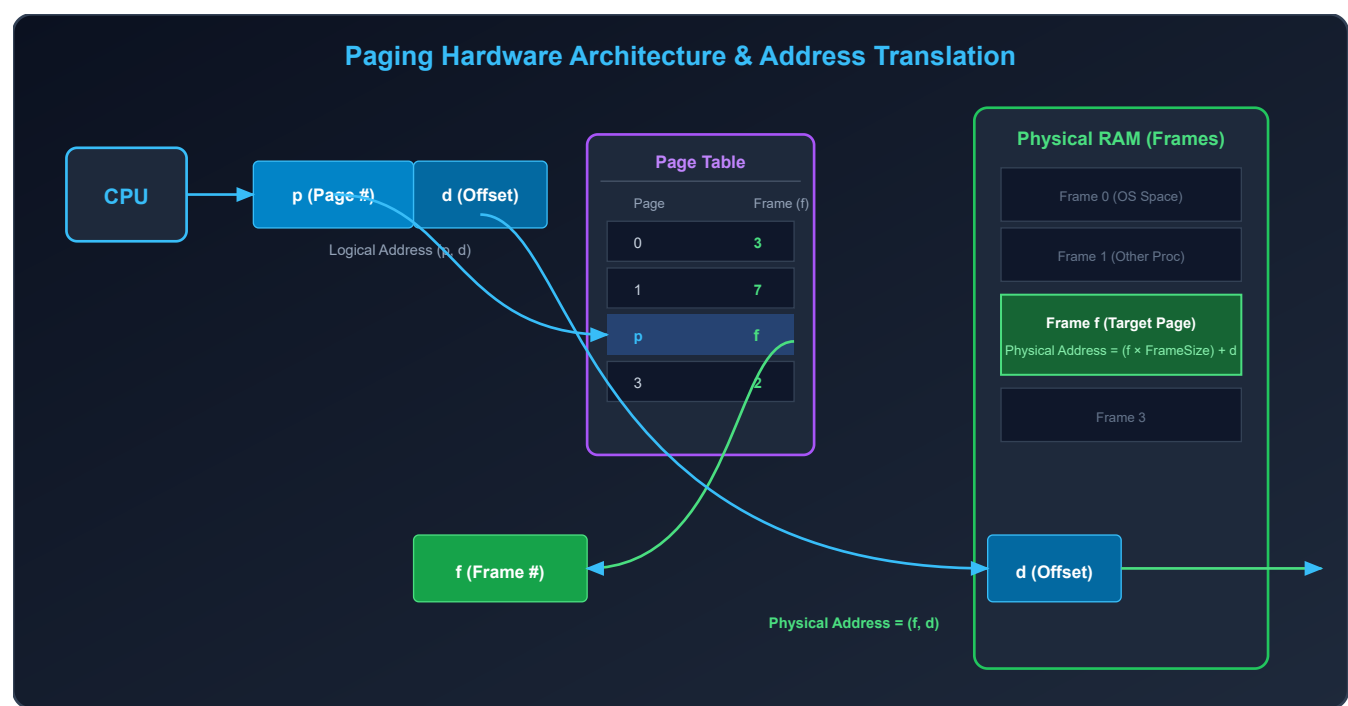
Module 04: Paging Architecture and Address Translation

1. Fundamental Paging Rules

Physical memory is split into fixed-size **Frames**; Logical space is split into identical-size **Pages**. Frame Size = Page Size = 2^n bytes. External fragmentation is completely eliminated!

2. Address Translation Formula

CPU generates (p, d). MMU accesses Page Table at index p to retrieve Frame Number f. Physical Address = (f × FrameSize) + d.



3. Key Formulas & Summary Table

Parameter	Mathematical Formula	Example (32-bit CPU, 4KB page)
Offset Bits (d)	$\log_2(\text{Page Size})$	$\log_2(4096) = 12$ bits
Page Bits (p)	$\text{Logical Bits (m)} - d$	$32 - 12 = 20$ bits
Total Pages	2^p	$2^{20} = 1\text{M}$ pages
Frame Bits (f)	$\log_2(\text{RAM Size} / \text{Frame Size})$	$\log_2(512\text{MB} / 4\text{KB}) = 17$ bits
Internal Frag	Worst: $\text{PageSize} - 1$; Avg: $\text{PageSize} / 2$	Worst: 4095 bytes; Avg: 2048 bytes

4. Solved Address Translation Numerical

Given: Page Size = 4096 bytes, Page 5 maps to Frame 12.

Logical Address: 20500.

1. Page Number $p = \text{floor}(20500 / 4096) = 5$.
2. Offset $d = 20500 \bmod 4096 = 20$.
3. Physical Address = $(12 \times 4096) + 20 = 49152 + 20 = \mathbf{49172}$.

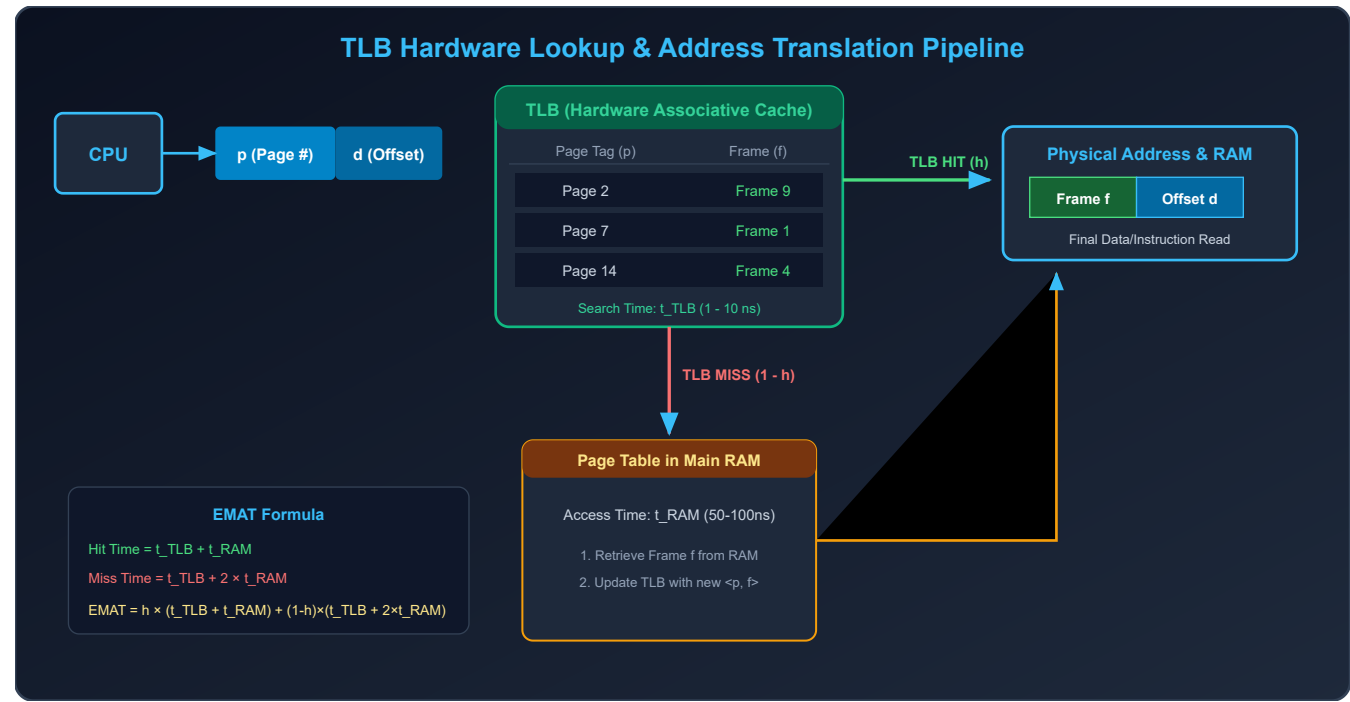
Module 05: Translation Lookaside Buffer (TLB) and EMAT

1. Pure Paging Overhead Problem

Pure paging me har instruction aur data access ke liye do memory references lagte hain: Page table lookup (100ns) + Data access (100ns) = 200ns. Is 100% latency penalty ko door karne ke liye CPU me **TLB (Translation Lookaside Buffer)** hardware cache add hota hai.

2. TLB Hit vs Miss & EMAT Formula

Condition	Probability	Access Formula	Example (t_TLB=20ns, t_RAM=100ns)
TLB Hit	h	$t_{TLB} + t_{RAM}$	$20 + 100 = 120\text{ ns}$
TLB Miss	$1 - h$	$t_{TLB} + 2 \times t_{RAM}$	$20 + 200 = 220\text{ ns}$
EMAT	Total	$h(t_{TLB} + t_{RAM}) + (1-h)(t_{TLB} + 2 \times t_{RAM})$	$h=0.8 \rightarrow \mathbf{140\text{ ns}} \mid h=0.98 \rightarrow \mathbf{122\text{ ns}}$



3. Protection & Valid-Invalid Bits

- **Valid (1):** Page process ke valid address space me map hai.
- **Invalid (0):** Illegal address access (Hardware trap / Segmentation Fault).
- **Permission bits:** Read (r), Write (w), Execute (x) enforce protection.

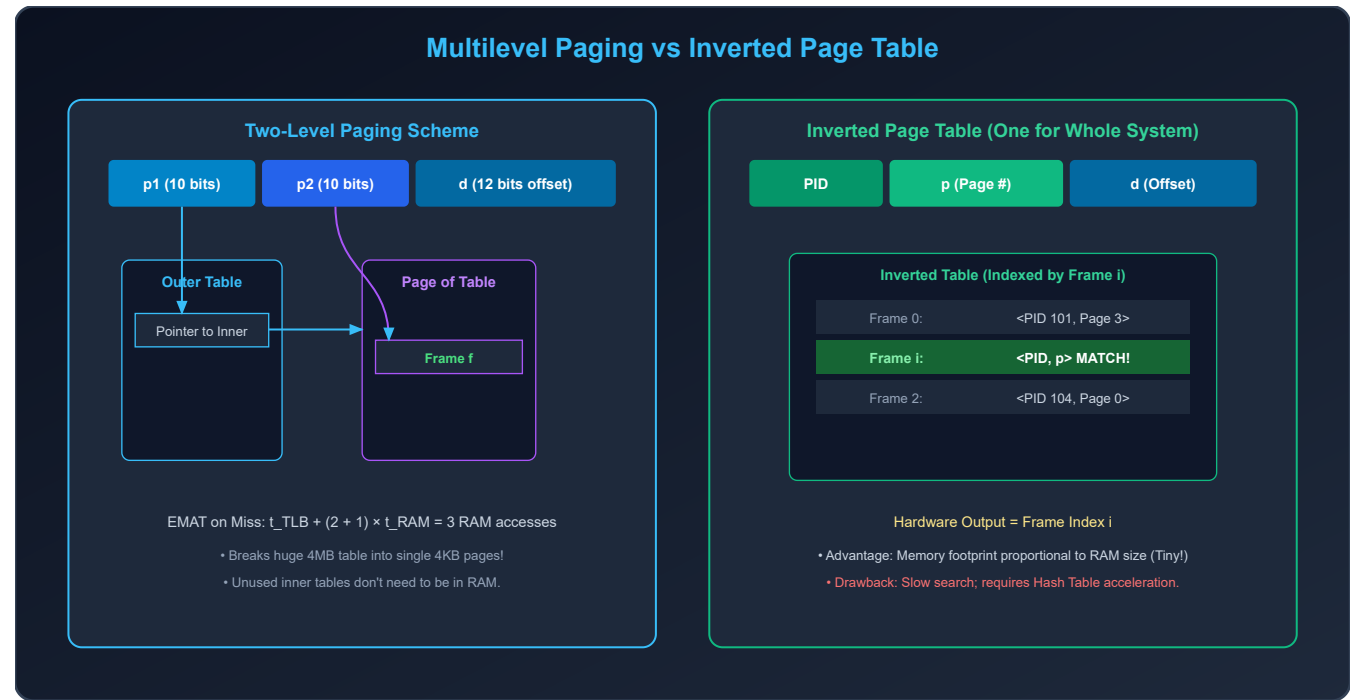
Module 06: Multilevel and Inverted Page Tables

1. Page Table Scaling Problem

32-bit system me 4KB pages ke sath har process ko 4MB ka contiguous page table chahiye. Multilevel paging is samasya ko solve karti hai page table ko khud page karke.

2. Two-Level Paging vs Inverted Page Table

Parameter	Hierarchical (Multilevel) Paging	Inverted Page Table
Number of Tables	Har process ke liye alag multi-level hierarchy	Poore system me strictly EK table
Indexing	Virtual Page Number ke dwara index	Physical Frame Number ke dwara index
Memory Overhead	High virtual space ke sath badhta hai	RAM size ke proportional (Extremely small)
Search Overhead	k-level me (k+1) RAM accesses per miss	Slow linear search (Hash table required)



3. Address Breakdown Formula

Two-Level 32-bit:

Logical Address = [p1 : 10 bits] | [p2 : 10 bits] | [d : 12 bits]

Inverted Address:

Logical Address = <PID, Page Number, Offset> → Matches Frame Index i in table.

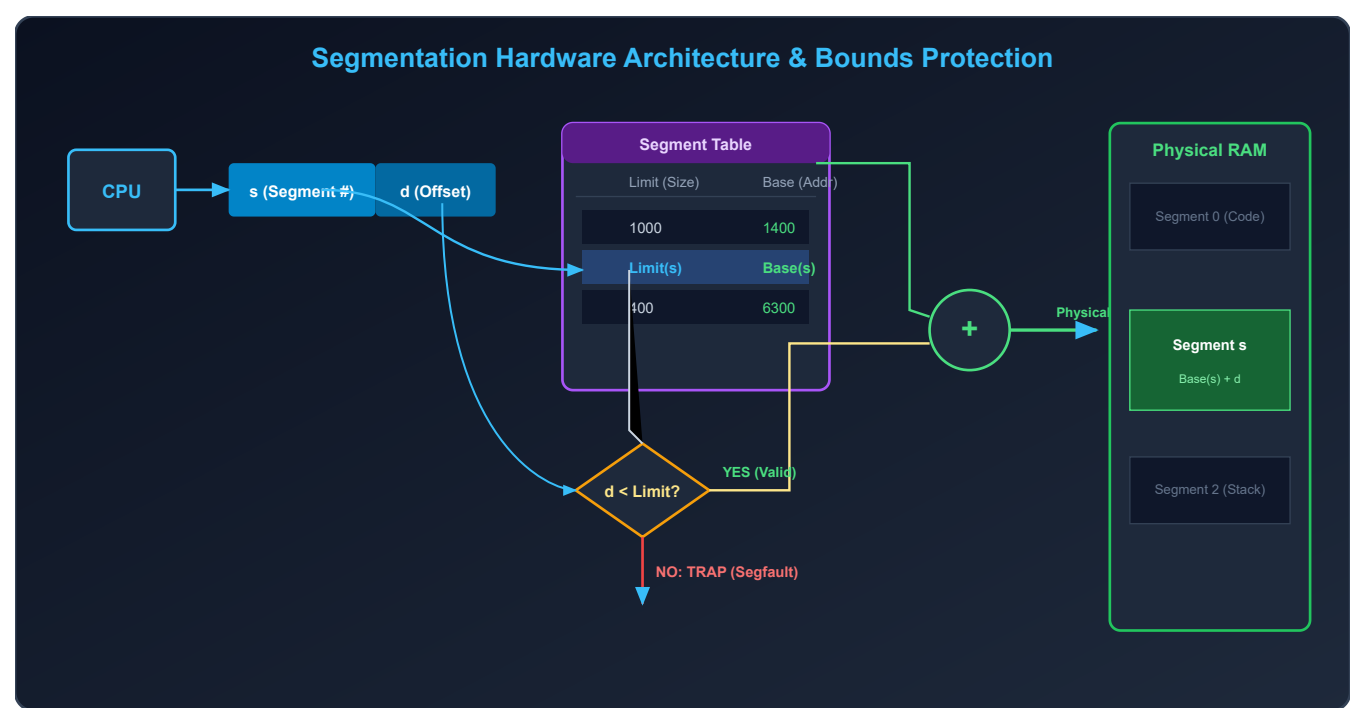
Module 07: Segmentation and Paged Segmentation

1. Programmer View vs OS View

Segmentation memory ko logical units (Code, Stack, Heap, Data) me organize karta hai. Address $\langle s, d \rangle$ hota hai jaha s = Segment number aur d = Offset.

2. Hardware Bounds Protection

Segment Table me do fields hote hain: **Base** (Physical start) aur **Limit** (Length). Agar $d \geq \text{Limit}$, hardware MMU ek **Trap (Segmentation Fault)** generate karta hai. Agar $d < \text{Limit}$, tab Physical Address = Base + d.



3. Paging vs Segmentation Comparison

Parameter	Paging	Segmentation
Block Size	Fixed (4 KB)	Variable
Decided by	Hardware / OS	Programmer / Compiler
Internal Fragmentation	Yes (in last page)	No (Zero)
External Fragmentation	No (Zero)	Yes
Address Calculation	Bit Concatenation	Base + Offset Addition

4. Solved Examples

- $\langle 0, 430 \rangle$: $430 < 600$ (Valid) $\rightarrow$ Base $219 + 430 = \mathbf{649}$.
- $\langle 2, 500 \rangle$: $500 \geq 100$ (Limit Exceeded) $\rightarrow$ **Trap: Segfault.**
- $\langle 3, 400 \rangle$: $400 < 580$ (Valid) $\rightarrow$ Base $1327 + 400 = \mathbf{1727}$.

Module 08: Virtual Memory and Demand Paging

AKTU BCS401

Unit 4

Virtual Memory Principles

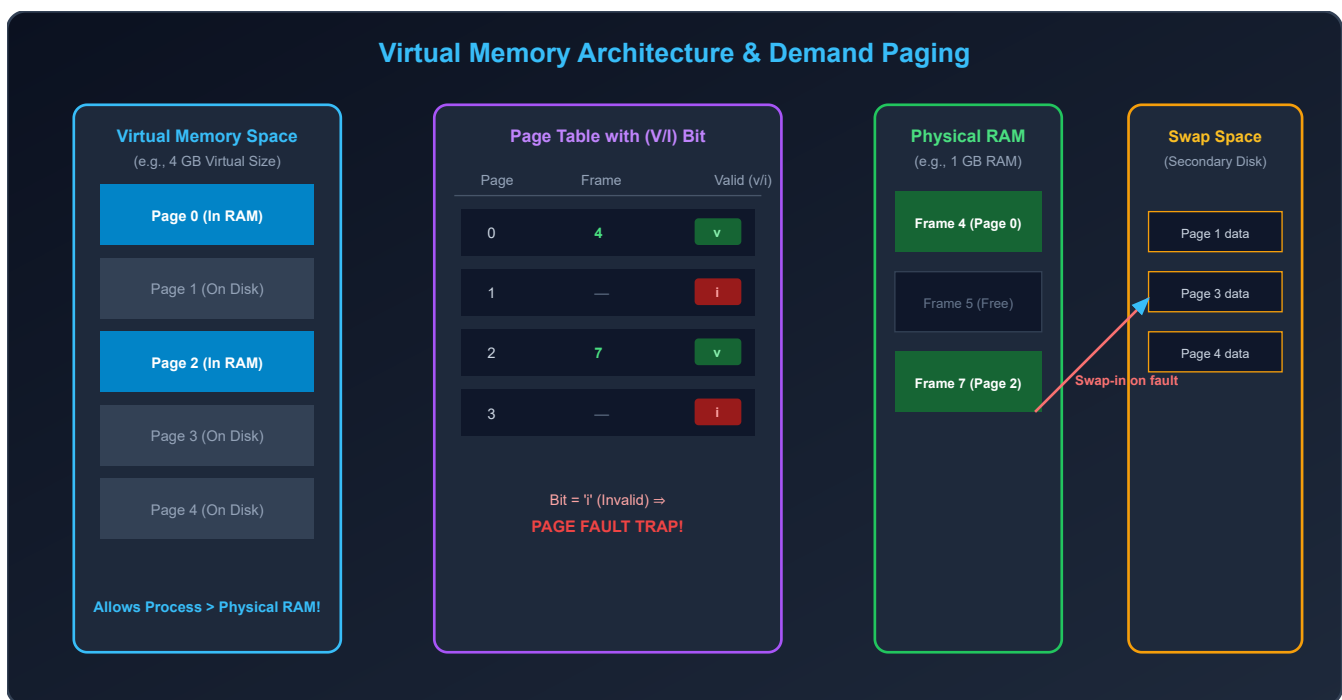
1. Virtual Memory Concept

Virtual memory user logical address space ko physical memory se alag karta hai, jisse process size physical RAM se bada ho sakta hai aur multiprogramming degree dramatically badh jaati hai.

2. Demand Paging & Valid-Invalid (V/I) Bit

Pages ko memory me tabhi load kiya jata hai jab unki demand ho (Lazy Swapper). Page Table me har entry me ek bit hota hai:

- **Valid (v):** Page RAM ke frame me present hai. Address translation direct hoti hai.
- **Invalid (i):** Page RAM me nahi hai balki disk par hai. MMU **Page Fault Trap** trigger karta hai.



3. Hardware Requirements

1. **Page table with V/I bit**
2. **Swap Space (Backing store)** on high-speed disk.
3. **Instruction Restartability:** Fault resolve hone ke baad trapped instruction ko exact usi state se restart karne ki capability.

Module 09: Page Fault Handling and EAT Performance

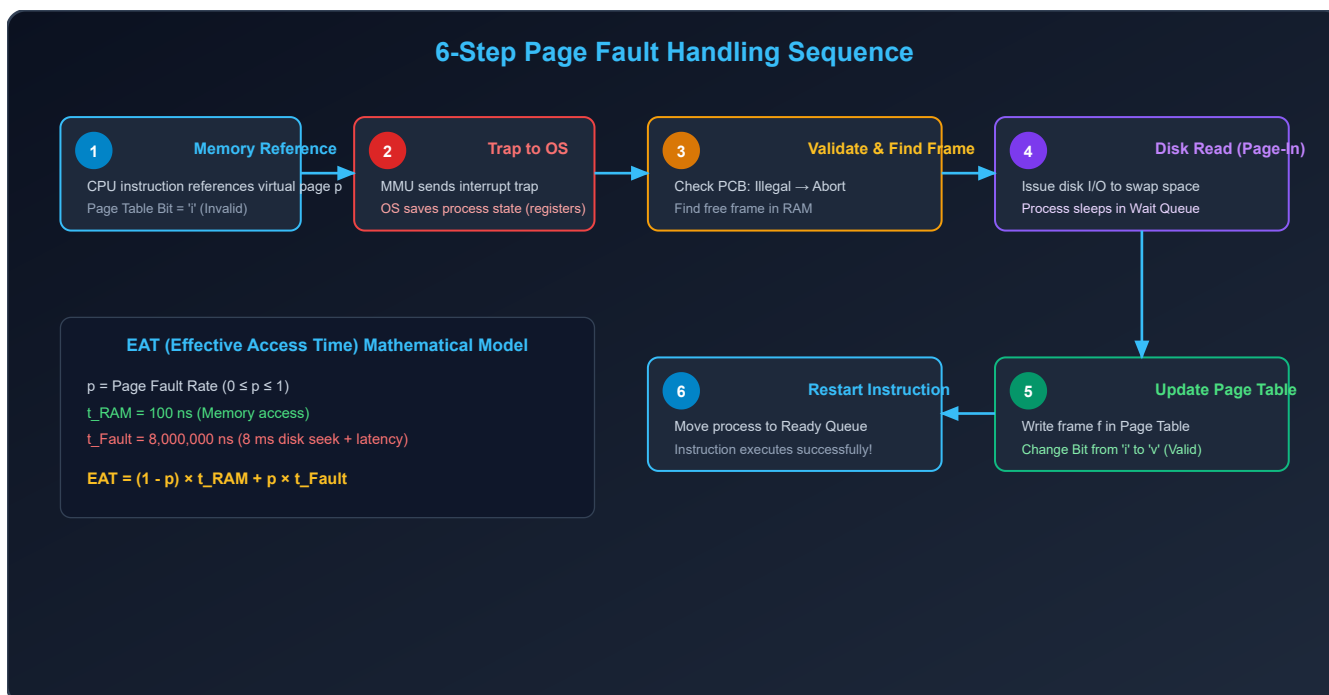
AKTU BCS401

Unit 4

Fault Recovery & EAT

1. Page Fault 6-Step Sequence

1. **Memory Reference:** CPU accesses invalid page → Hardware MMU trap.
2. **Trap to OS:** Kernel saves process state to PCB.
3. **Legality Check:** If legal, find free frame; if illegal, abort process.
4. **Disk I/O Swap-In:** Read missing page from swap space into RAM frame.
5. **Update Page Table:** Enter frame number, flip bit to 'v' (Valid).
6. **Restart Instruction:** Trapped instruction restarts seamlessly.



2. Effective Access Time (EAT) Formula

$$EAT = (1 - p) \times t_{RAM} + p \times (\text{Page Fault Service Time})$$

Solved Example: $t_{RAM} = 200 \text{ ns}$, Fault Service Time = 8 ms.

10% max slowdown ke liye: $p \leq 20 / 8,000,000 = 1 / 400,000$ (0.00025%).

Har 4 lakh memory accesses me strictly 1 se kam page fault aana chahiye!

Module 10: FIFO Page Replacement and Belady's Anomaly

1. FIFO Algorithm & Dirty Bit

FIFO memory me sabse pehle enter karne wale page ko replace karta hai. Dirty Bit (0/1) indicate karta hai ki page modify hua hai ya nahi; clean page evict karte samay disk write save ho jata hai.

2. Belady's Anomaly Proof

Normally frames badhane se faults kam hote hain, lekin FIFO me frames 3 se 4 karne par faults 9 se badhkar 10 ho jate hain!

Belady's Anomaly: FIFO 3 Frames vs 4 Frames Proof

Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO with 3 Frames (9 Page Faults)

Initial: [Empty]

Refs 1, 2, 3: Frames = [1, 2, 3] → 3 Faults

Ref 4: Replace 1 → [4, 2, 3] → Fault 4

Ref 1: Replace 2 → [4, 1, 3] → Fault 5

Ref 2: Replace 3 → [4, 1, 2] → Fault 6

Ref 5: Replace 4 → [5, 1, 2] → Fault 7

Ref 1: HIT! Frames = [5, 1, 2]

Ref 2: HIT! Frames = [5, 1, 2]

Ref 3: Replace 1 → [5, 3, 2] → Fault 8

Ref 4: Replace 2 → [5, 3, 4] → Fault 9

Ref 5: HIT! Frames = [5, 3, 4]

Total Page Faults = 9

FIFO with 4 Frames (10 Page Faults!)

Refs 1, 2, 3, 4: Frames = [1, 2, 3, 4] → 4 Faults

Ref 1: HIT! Frames = [1, 2, 3, 4]

Ref 2: HIT! Frames = [1, 2, 3, 4]

Ref 5: Replace 1 → [5, 2, 3, 4] → Fault 5

Ref 1: Replace 2 → [5, 1, 3, 4] → Fault 6

Ref 2: Replace 3 → [5, 1, 2, 4] → Fault 7

Ref 3: Replace 4 → [5, 1, 2, 3] → Fault 8

Ref 4: Replace 5 → [4, 1, 2, 3] → Fault 9

Ref 5: Replace 1 → [4, 5, 2, 3] → Fault 10

Total Page Faults = 10 (Faults BADH GAYE!)

Frames 3 → 4 badhane par Faults 9 → 10 ho gaye!
• FIFO is NOT a Stack Algorithm.

3. Summary Comparison

Why it happens: FIFO is NOT a Stack Algorithm ($S_n \not\subseteq S_{n+1}$).

Stack Algorithms: LRU and Optimal algorithms strictly follow subset property and NEVER suffer from Belady's Anomaly!

Module 11: Optimal and LRU Page Replacement

1. Optimal (OPT / MIN) vs LRU

Parameter	Optimal (OPT)	Least Recently Used (LRU)
Direction	Looks Forward into future	Looks Backward into past
Fault Rate	Minimum possible benchmark	Very close to Optimal
Belady Anomaly	Never occurs (Stack algo)	Never occurs (Stack algo)
Feasibility	Unrealizable in real-time OS	Implementable via Counter or Stack

Optimal (OPT) vs Least Recently Used (LRU)

String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (Frames = 3)

Optimal (OPT / MIN) Algorithm

Looks FORWARD into the future

- Rule: Us page ko replace karo jo future me **longest period of time tak use nahi hoga.**
- Minimum possible page fault rate guarantee karta hai.
- Practicality: Cannot be implemented in general OS (Requires future oracle knowledge).
- Benchmark: Dusre algorithms ki efficiency measure karne ke liye theoretical baseline ka kaam karta hai.

Total Faults on String = 9 Page Faults

Least Recently Used (LRU) Algorithm

Looks BACKWARD into the past

- Rule: Us page ko replace karo jo past me **longest period of time se access nahi hua hai.**
- OPT ka best practical approximation hai.
- Stack Algorithm: Belady's Anomaly se 100% immune!
- Implementation Methods:
 1. Counter (Logical Clock) - Search min counter on fault
 2. Doubly Linked Stack - Most recent to top, bottom = victim

Total Faults on String = 12 Page Faults

2. 20-Reference Trace Result (3 Frames)

- **Optimal Faults: 9 Faults** (Hit Ratio = 55%)
- **LRU Faults: 12 Faults** (Hit Ratio = 40%)
- **FIFO Faults:** 15 Faults (Suffers from lack of locality).

Module 12: Counting Algorithms and Clock Replacement

1. Second Chance (Clock) Replacement

Circular queue with a clock pointer. If Reference bit $R=0 \rightarrow$ Replace. If $R=1 \rightarrow$ Clear $R=0$, give second chance, advance pointer.

2. Enhanced Second Chance 4-Class Model

Class <R, M>	Meaning	Action
(0, 0)	Not recently used, Clean (not modified)	Best candidate for replacement
(0, 1)	Not recently used, Dirty (modified)	Requires disk write before page-out
(1, 0)	Recently used, Clean	Likely in active working set
(1, 1)	Recently used, Dirty	Worst candidate

Second Chance (Clock) Replacement

Clock Pointer inspects Frame

- If $R == 0 \rightarrow$ Replace this page (Victim found!)
- If $R == 1 \rightarrow$ Set $R = 0$, advance pointer (Second Chance!)

Enhanced Second Chance & Counting Algos

Enhanced 4-Class Classification <Reference, Modify>:

- Class 1 (0, 0):** Not used, Not dirty $\rightarrow$ BEST VICTIM!
- Class 2 (0, 1):** Not used, Dirty $\rightarrow$ Needs disk write
- Class 3 (1, 0):** Recently used, Clean $\rightarrow$ Will be used soon
- Class 4 (1, 1):** Recently used, Dirty $\rightarrow$ WORST CANDIDATE

Counting-Based Algorithms:

- LFU (Least Frequently Used):**
Page with smallest reference count is replaced.
- MFU (Most Frequently Used):**
Page with highest count is replaced (newly brought page has count 0).

3. LFU vs MFU

- **LFU:** Smallest reference count replaced.
- **MFU:** Highest reference count replaced (protects new arrivals).

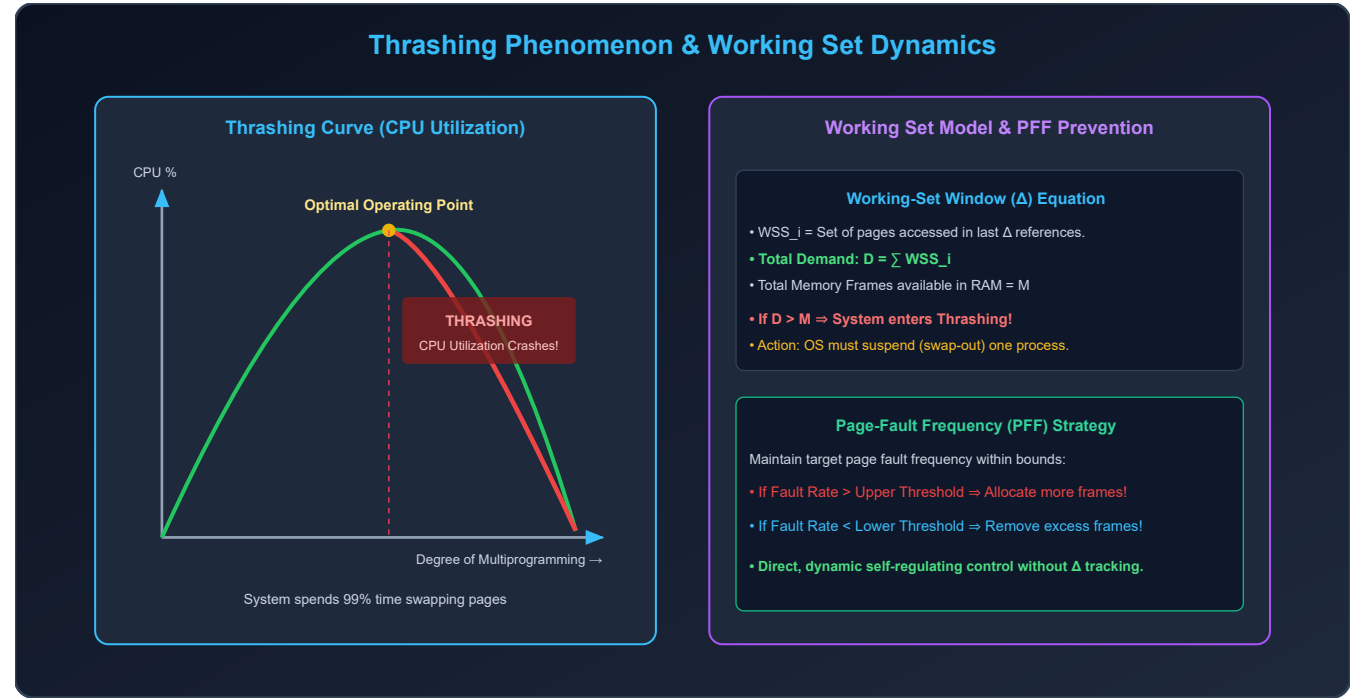
Module 13: Thrashing and Working-Set Model

1. Thrashing & Vicious Cycle

Jab system instruction execute karne ke bajaye swapping me zyada time bitata hai to CPU utilization 0% ho jaati hai aur disk 100% busy ho jaati hai.

2. Working-Set Model vs PFF

Method	Key Formula / Parameter	Prevention Mechanism
Working-Set Model	$D = \sum WSS_i > M$	If demand $D >$ total frames M , swap-out a process
Page-Fault Frequency	Upper & Lower Thresholds	Directly adds frames if rate $>$ upper; reclaims if $<$ lower



3. Key Takeaways

- Thrashing is caused by over-allocating degree of multiprogramming.
- Solution: Local page replacement, working set monitoring, suspending processes.

Module 14: Locality of Reference and Cache Organization

1. Principle of Locality

- **Temporal Locality:** Same location accessed repeatedly soon (Loops, counters).
- **Spatial Locality:** Nearby consecutive memory locations accessed soon (Arrays, sequential instructions).

2. Cache Mapping Techniques

Scheme	Mapping Rule	Pros & Cons
Direct	Line = Block mod N	Fastest & cheapest; high conflict misses
Fully Associative	Any block in any line	No conflict misses; expensive hardware comparators
Set-Associative	Set = Block mod NumSets	Best industry compromise between cost & performance

Locality of Reference & Cache Memory Organization

Principle of Locality

1. Temporal Locality (Locality in Time)

- Concept: Jo memory location abhi access hui hai, woh near future me DOBARA access hone ki **high probability rakhti hai.**
- Examples: Loops (for, while), loop index counters, subroutine/function code, top of stack.

2. Spatial Locality (Locality in Space)

- Concept: Jo location abhi access hui hai, uske AAS-PAAS (neighboring) ke memory locations **soon access hone ki high probability rakhte hain.**
- Examples: Array sequential traversal, continuous code instruction stream, consecutive struct fields.

Cache Mapping Architectures

1. Direct Mapping

Block i maps to Cache Line = (i mod N). Fast & simple hardware.

Conflict Misses: High when multiple blocks map to same line.

2. Fully Associative Mapping

Any block can go into ANY cache line. Zero conflict misses!

Costly: Requires parallel hardware comparators for every line.

3. k-Way Set-Associative (Industry Standard)

Cache is divided into Sets; each Set contains k lines.

Block maps to Set = (Block mod NumberOfSets); inside set any line.

AMAT = Hit_Time + (Miss_Rate × Miss_Penalty)

3. AMAT Formula

AMAT = Hit_Time + (Miss_Rate × Miss_Penalty)

Module 15: AKTU PYQs and Technical Interview Cheatsheet

1. Master Formula Sheet

Concept	Mathematical Formula
TLB EMAT	$h \times (t_{TLB} + t_{RAM}) + (1 - h) \times (t_{TLB} + 2 \times t_{RAM})$
Demand Paging EAT	$(1 - p) \times t_{RAM} + p \times t_{Fault}$
Segmentation	Physical = Base + Offset (If Offset < Limit)
Working Set Thrashing	Demand $D = \sum WSS_i > \text{Physical Memory } M$

Unit 4 High-Yield Formulas & Interview Cheatsheet

1. Paging & Address Translation

- Page Size = Frame Size = 2^n bytes $\Rightarrow$ Offset $d = n$ bits
- Page Number $p = \text{Logical Bits } (m) - n \text{ bits}$
- **Physical Address = (Frame Number $f \times \text{FrameSize}$) + d**
- Pure Paging Latency = $2 \times t_{RAM}$ (Double access penalty)
- Internal Frag = Worst (PageSize - 1), Average (PageSize / 2)

2. TLB & Effective Access Time

- Hit Time = $t_{TLB} + t_{RAM}$
- Miss Time = $t_{TLB} + 2 \times t_{RAM}$
- **EMAT = $h \times (t_{TLB} + t_{RAM}) + (1-h) \times (t_{TLB} + 2 \times t_{RAM})$**
- Two-Level Miss = $t_{TLB} + 3 \times t_{RAM}$
- Demand Paging EAT = $(1-p) \times t_{RAM} + p \times (t_{Fault})$

3. Page Replacement Rules

- FIFO: Replaces oldest; suffers Belady's Anomaly
- **OPT: Longest future use; minimum faults (theoretical)**
- **LRU: Least recent past use; Stack algorithm (NO anomaly)**
- Clock (Second Chance): $R=0$ replace; $R=1$ reset to 0
- Enhanced Clock: (0,0) best > (0,1) > (1,0) > (1,1) worst
- LFU/MFU: Smallest count / Largest count replaced

4. Segmentation & Thrashing

- Segment Address = $\langle s, d \rangle$ | If $d \geq \text{Limit} \Rightarrow$ Trap (Segfault)
- **If $d < \text{Limit} \Rightarrow \text{Physical Address} = \text{Base}(s) + d$**
- Thrashing: CPU thrashing when Demand $D = \sum WSS_i > M$
- PFF Strategy: Dynamic threshold frame allocation
- **Compaction: Requires Execution Time Dynamic Relocation**
- Cache AMAT = Hit_Time + Miss_Rate $\times$ Miss_Penalty

2. High-Yield Interview Topics

- **ASID Tagging vs TLB Flush:** Context switch par ASID tag use karke TLB flush avoid kiya jata hai.
- **Copy-On-Write (COW):** fork() par parent aur child shared read-only frames use karte hain; memory write par single page copy hota hai.
- **Belady's Anomaly:** FIFO suffers because it is not a stack algorithm ($S_n \not\subseteq S_{n+1}$).